

Day 14 — Jenkins + Docker

Status: ■ Not started

Objectives

- Use Docker containers as build agents (ephemeral, clean environments)
- Build and push Docker images from a pipeline
- Understand Docker-in-Docker (DinD) vs mounting the host socket
- Deploy a container as the final pipeline stage

Lab Steps

Install Docker Pipeline plugin

- Manage Jenkins → Plugins → search "Docker Pipeline"

Use a Docker agent for a build

- agent { docker { image 'node:20-alpine' } }
- Confirm the build runs isolated, with no leftover state between runs

Build and push an image

- Use `jenkinsfiles/docker/Jenkinsfile` from this repo
- Configure Docker Hub credentials in Jenkins
- Build, tag with `${BUILD_NUMBER}` and `latest`, push both tags

Docker socket vs Docker-in-Docker

- Compare mounting `/var/run/docker.sock` into the Jenkins container (simpler, shares host daemon)

vs running a full DinD sidecar (more isolated, more complex)

- Discuss security implications of socket mounting

Deploy stage

- Add a `when { branch 'main' }` deploy stage that stops/removes the old container and runs the new image

Key Takeaways

- Docker agents give reproducible, disposable build environments
- Mounting the Docker socket is the common (if imperfect) approach for Jenkins-in-Docker setups
- Always tag images with a unique build identifier, not just `latest`

Checklist

- ☐ Docker Pipeline plugin installed
- ☐ Pipeline runs inside a Docker agent successfully
- ☐ Image built, tagged, and pushed to a registry
- ☐ Deploy stage runs only on main branch

Day 14 — Interview Questions: Jenkins + Docker

What are the benefits of using Docker containers as Jenkins build agents?

Explain "Docker-outside-of-Docker" (socket mounting) vs "Docker-in-Docker" (DinD). What are the trade-offs of each?

Why is mounting `/var/run/docker.sock` into a Jenkins container considered a security consideration, not just a convenience?

How do you authenticate to a private Docker registry from a pipeline without hardcoding credentials?

What's a sensible image tagging strategy for CI-built images, and why avoid relying solely on `latest`?

How would you keep a Jenkins host from running out of disk space due to Docker image buildup?

Describe a deploy stage that safely replaces a running container with a newly built image.

How would you scan a built image for vulnerabilities as part of the pipeline?

<details><summary>Answer notes</summary>

- Q3: Mounting the socket effectively gives root-equivalent access to the host — any job that can run `docker` commands can escape its "container" isolation. Mitigate with RBAC on who can edit pipeline code, and consider dedicated build hosts for untrusted jobs.
- Q5: Tag with a unique, traceable identifier (build number, git SHA) in addition to `latest`, so any deployed version can be pinned and rolled back precisely.
- Q6: Schedule `docker system prune` periodically, or use ephemeral build agents that are destroyed after each build.

</details>